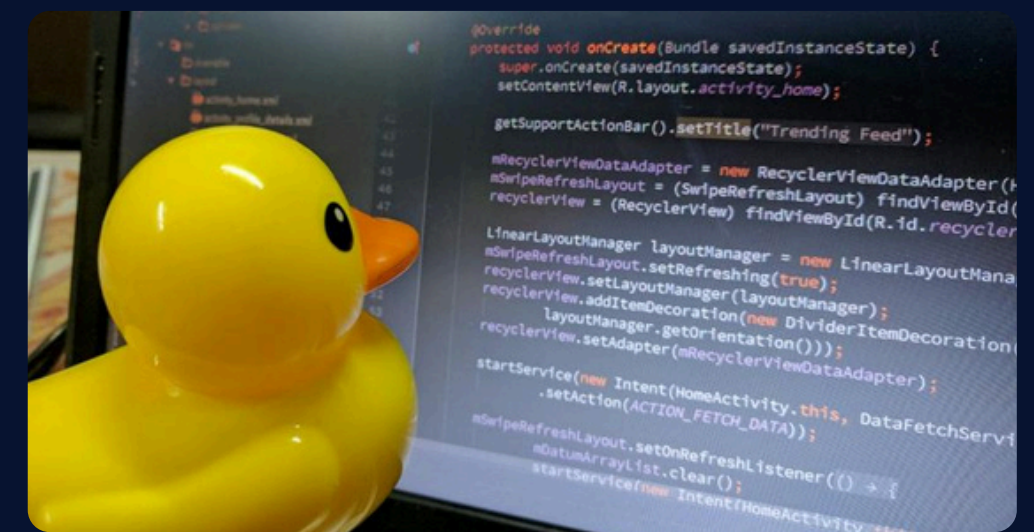




Linear Regression



Session -> 5



Intro to Linear Regression

Machine Learning Types

```
graph TD; A[Machine Learning Types] --> B[Supervised Learning]; A --> C[Unsupervised Learning]; A --> D[Reinforcement Learning]; B --> E[Resgression]; B --> F[Classification]; C --> G[Clustering]; C --> H[Dimensionality Reduction];
```

Supervised Learning

Resgression

Classification

Unsupervised Learning

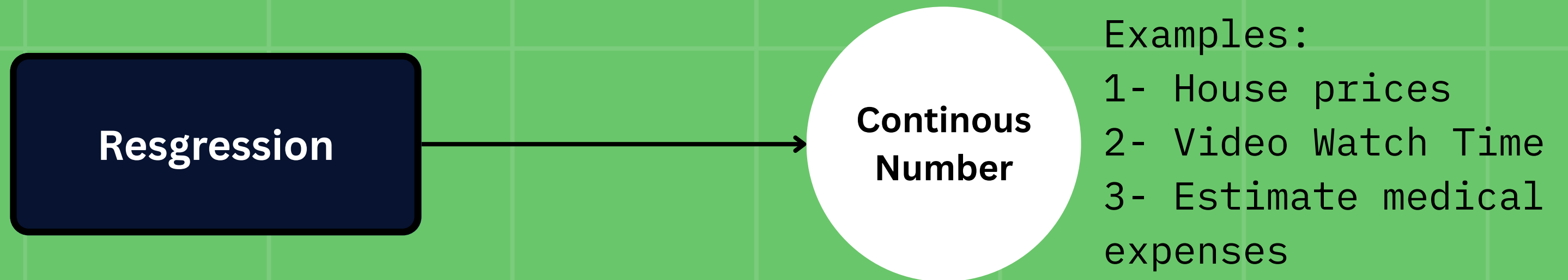
Clustering

**Dimensionality
Reduction**

Reinforcement Learning

What is Regression?

- The goal of regression is to create a predictive model that can make continuous predictions or estimates, as opposed to classification, which deals with discrete categories or classes.



Independant **VS** dependant Variables

- Regression in machine learning is used to predict numerical values by modeling the relationship between a dependent variable (target) and one or more independent variables (features or predictors)."

Target (Dependent variable) = What you want to predict.

Feature (Independent variables) = The inputs used to make the prediction.

Independant **VS** dependant Variables

Example: Car Price Prediction

- Target (Dependent Variable):
= Car price (in USD)
- Features (Independent Variables):
 - = Engine size (liters)
 - = Year of manufacture
 - = Brand
 - = Fuel type

Predicting House Prices

- Imagine you're trying to predict a house price based of the number of rooms.

Number of Rooms	House Price
1	150
2	200
3	250
4	?
5	350
6	400
7	450

From the provided data, we can observe that as the number of rooms **increases**, the price of the house **increases** too

Predicting House Prices

- You may noticed that when 1 room is added, the price increases by 50\$ - and the base price (the number we start by when rooms = 0) is 100\$
- What we have just did by observing the pattern is what "Linear Regression" do

new price = 50 * number of rooms + 100

$$y = w * x + b$$

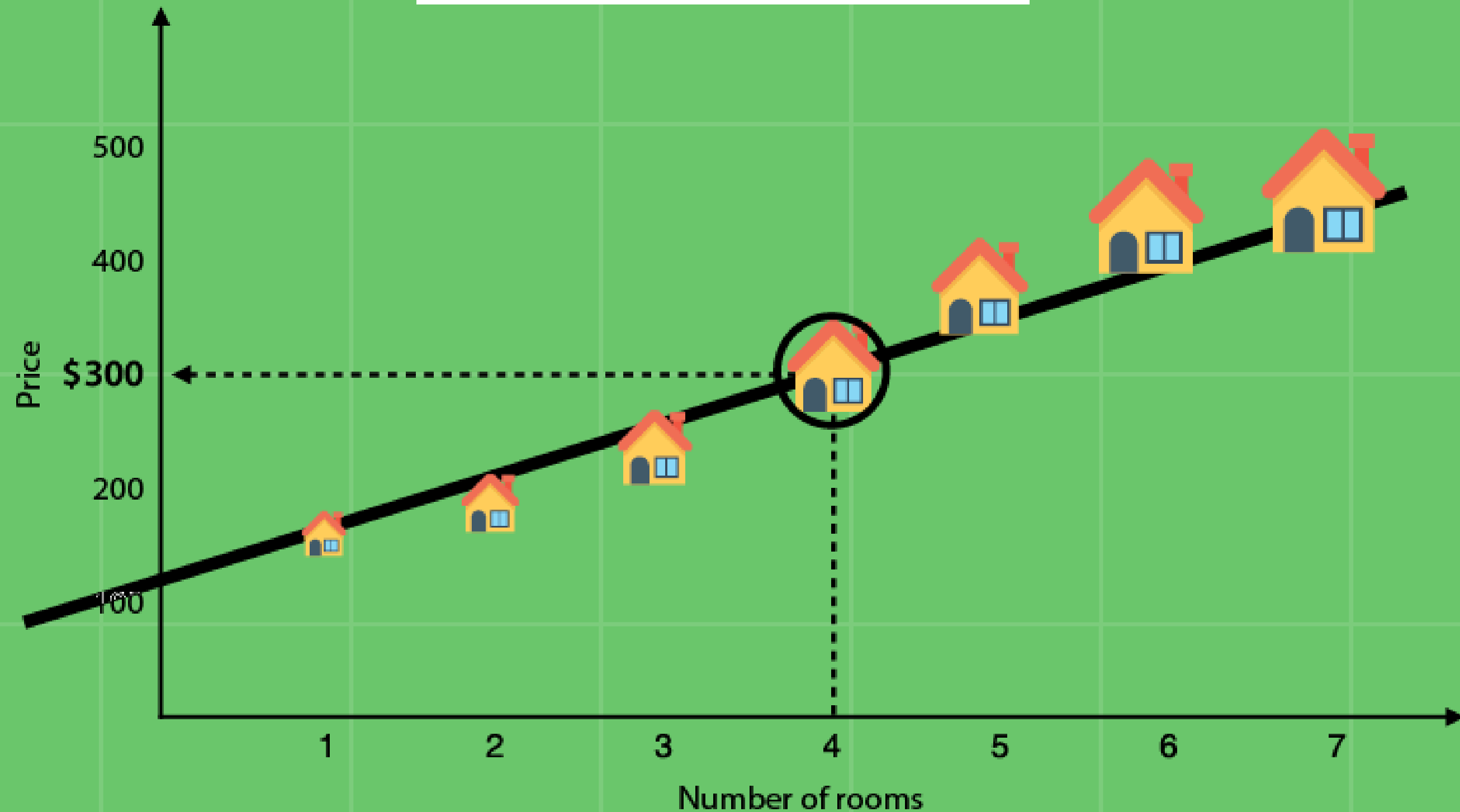
Linear Regression Equation

$$y = w * x + b$$

- **y** → The value we want to predict. (Target)
- **x** → The input feature or independent variable that we use to make the prediction. (Feature)
- **w** → This is the slope of the line which tells us how much y changes when x changes by 1 unit. (Weight)
- **b** → This is the bias or intercept. It represents the value of y when x=0 (bias)

Linear Regression Equation

$$y = m * x + b$$



House Pricing with harder pattern

- This is a more complicated dataset:

Number of Rooms	House Price
1	155
2	197
3	244
4	?
5	356
6	407
7	448

Notice that prices don't follow a good pattern, how can we compute the w ?

House Pricing with multiple features

- This is a more complicated dataset:

Number of Rooms	area	House Price
1	95	155
2	95	197
3	100	244
4	160	?
5	170	356
6	180	407
7	180	448

Here when we added one extra feature, how can we get the weights?

$$y = w1 * x1 + w2 * x2 + b$$

Why Use Linear Regression in Machine Learning?

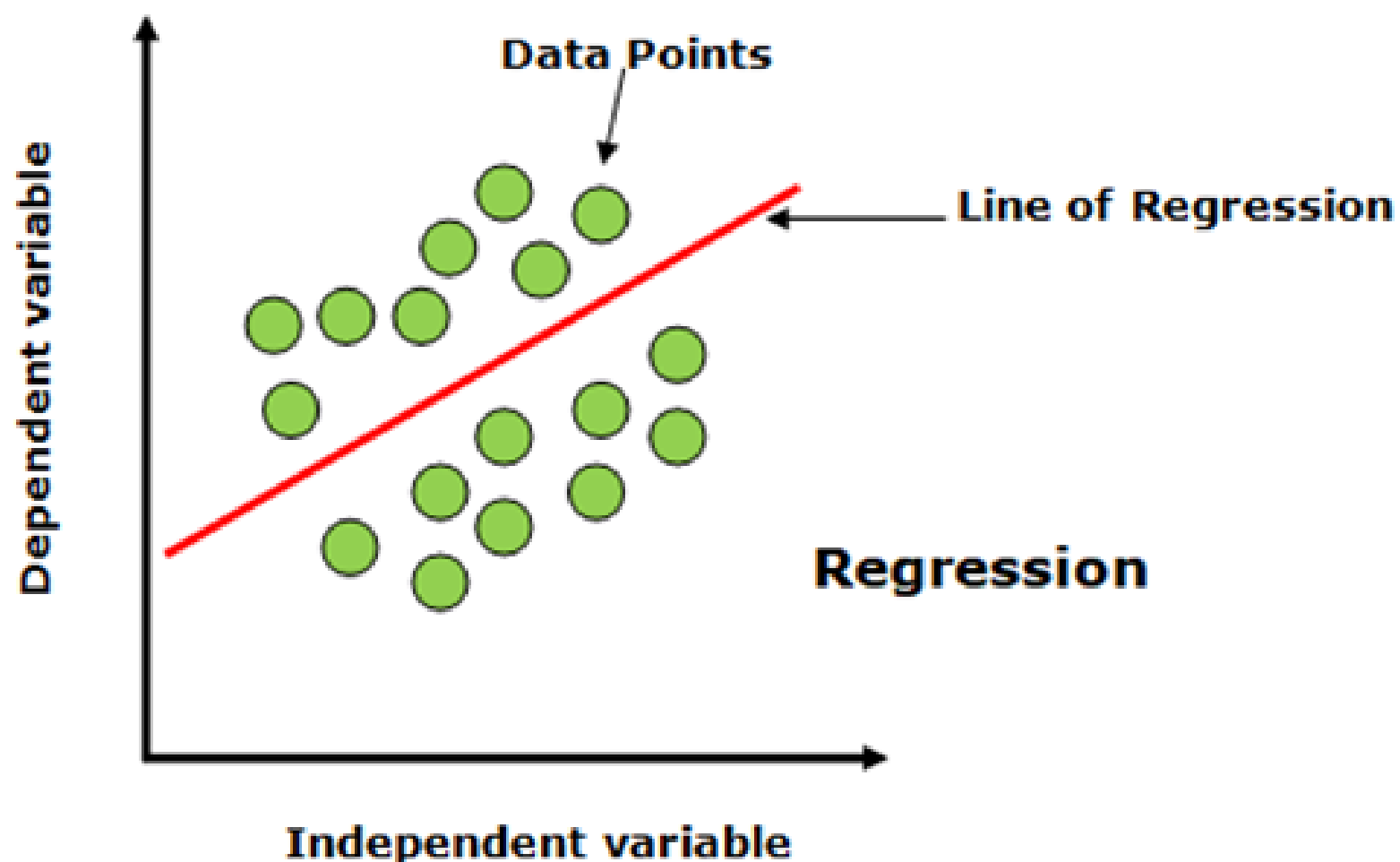
- Linear regression in machine learning is used to find the best-fitting line through data, especially **when the slope isn't fixed or there are multiple features**. It learns the relationship between inputs and output by minimizing error. This helps us make predictions, even with noisy or complex data.



Linear Regression in Machine Learning

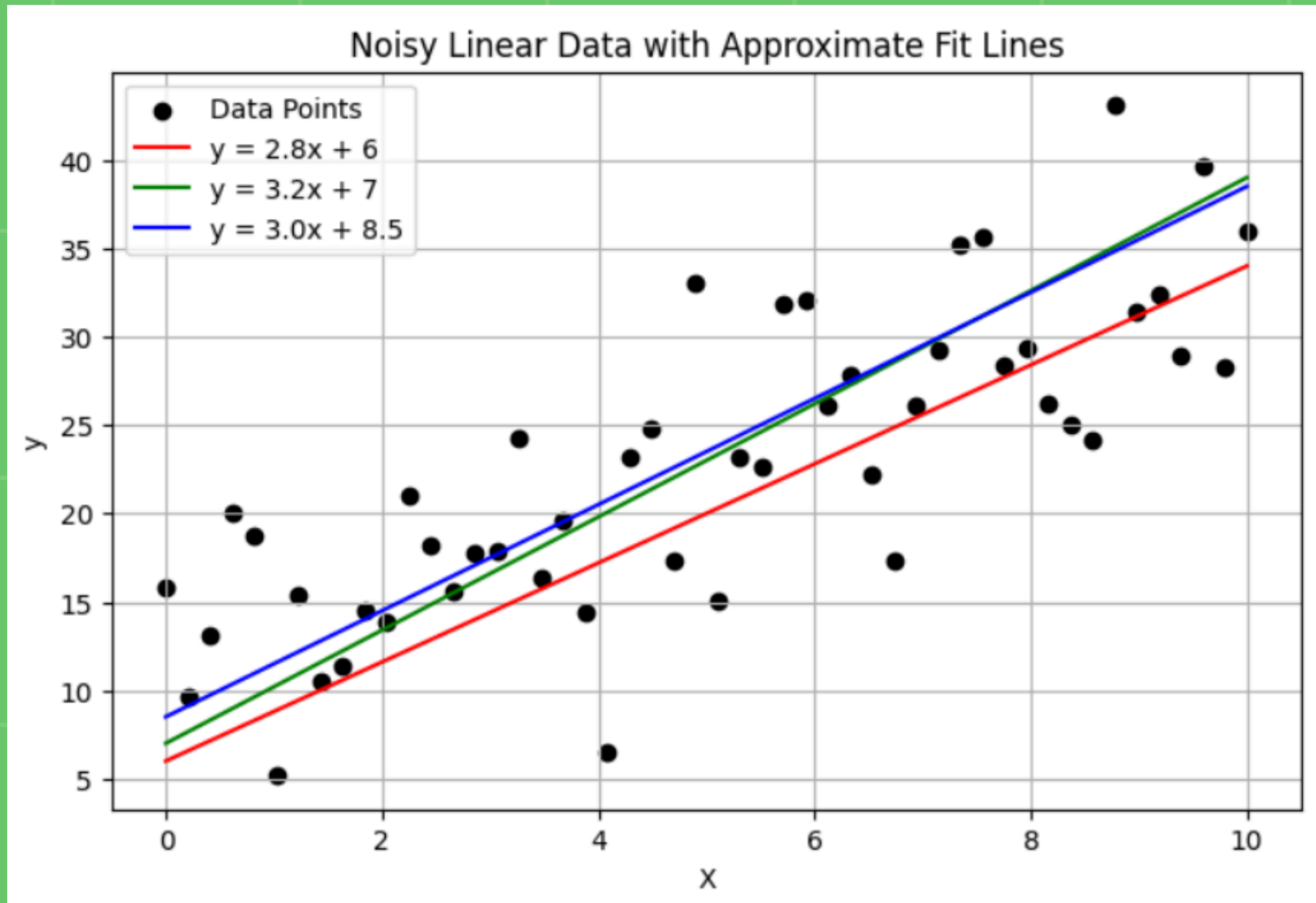
What is Linear Regression?

- Linear regression is a method used to find the best-fitting straight line that represents the linear relationship between one or more input variables (features) and an output (target).



Best line =
the right **weight** & **bias**

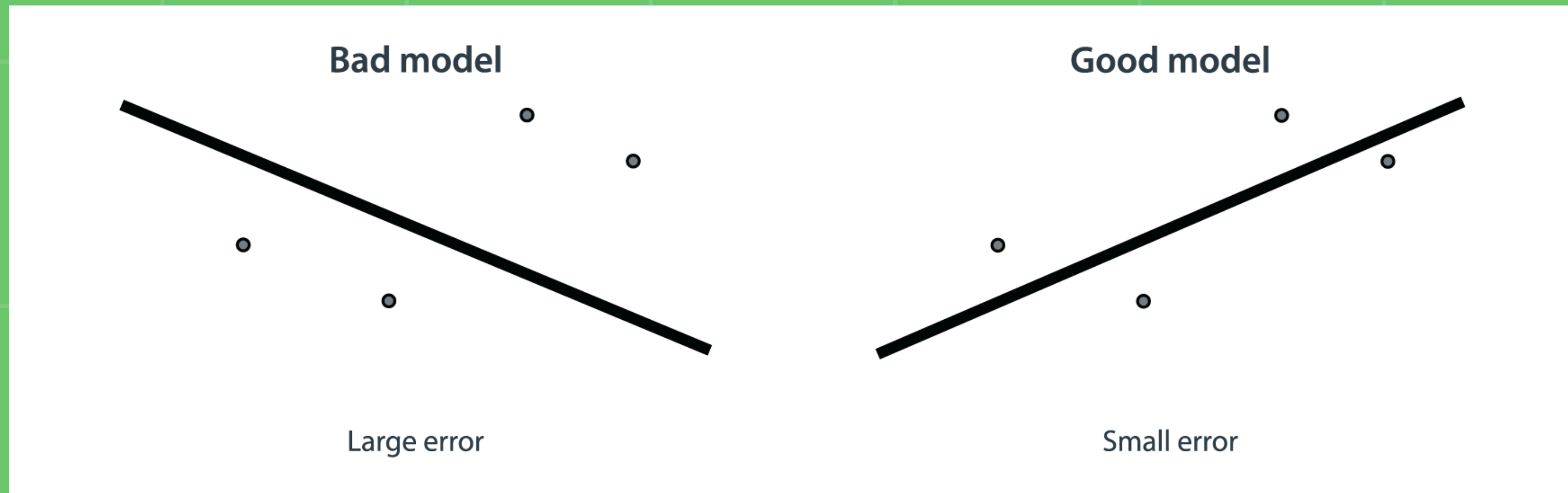
What is the best line?



The error function

(How do we measure our results?)

- The error function is a metric that tells us how our model is doing, for example: it tells us that the left figure is the bad model and the right is the good one based on the difference between the actual value and the predicted value for each point:



The error function- Absolute Error

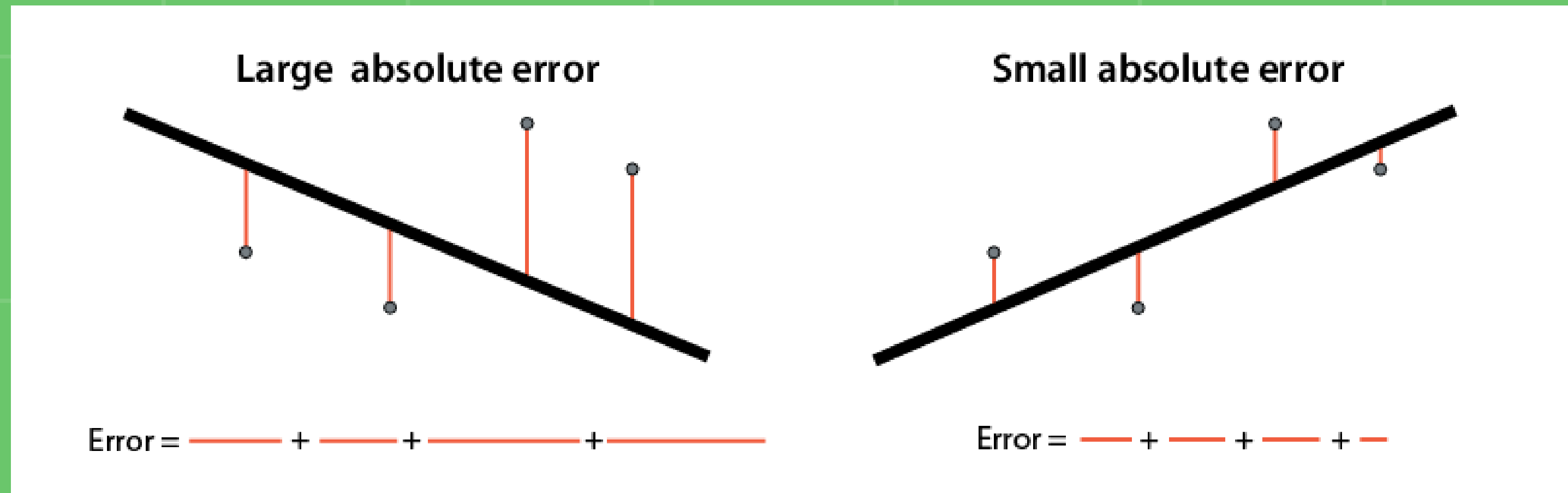
- In linear regression, the error is the difference between the predicted value and the actual value for each data point:

$$\text{Error} = y'(i) - y(i)$$

Where:

- $y'(i)$: is the predicted value.
- $y(i)$: is the actual (true) value.

The error function- Absolute Error



$$\text{Error} = y'(i) - y(i)$$

The error function- Square Error

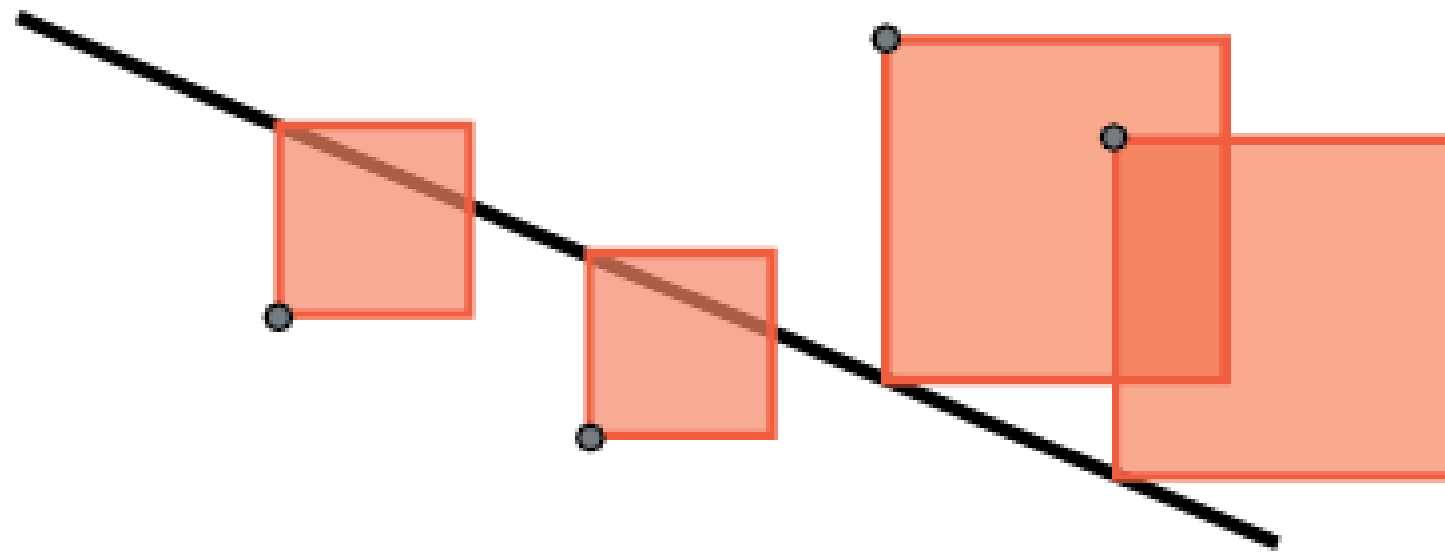
- The square error is very similar to the absolute error, except instead of taking the absolute value of the difference between the label and the predicted label, we take the square.
- This always turns the number into a positive number

$$\text{Error} = (y'(i) - y(i))^2$$

Squared Error is used commonly in practice than absolute error

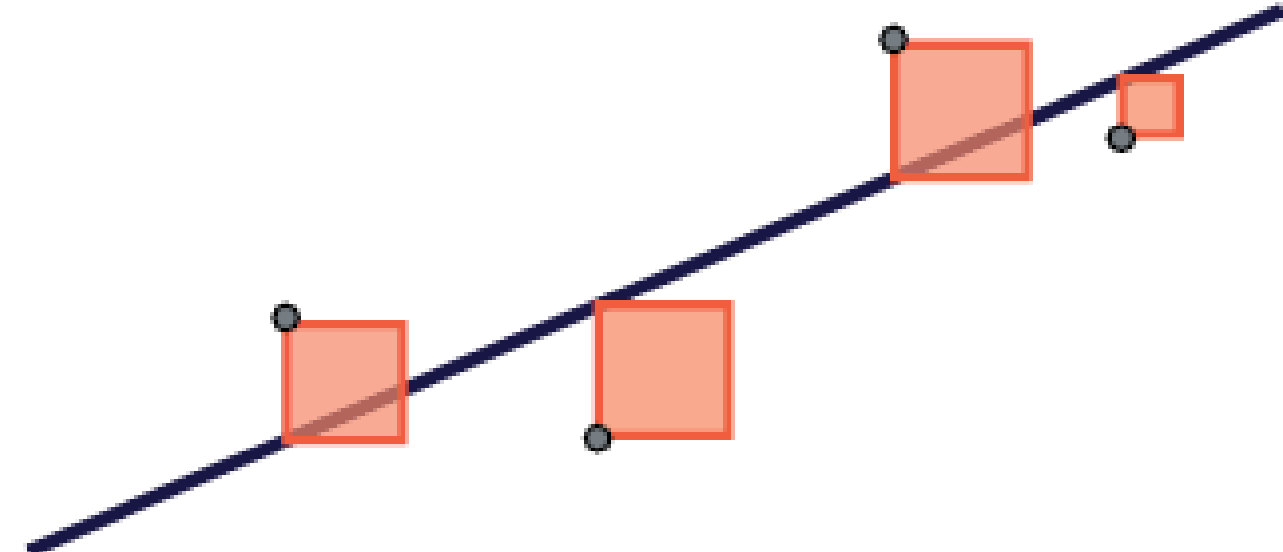
The error function- Square Error

Large square error



$$\text{Error} = \text{[small square]} + \text{[small square]} + \text{[large square]} + \text{[large square]}$$

Small square error



$$\text{Error} = \text{[small square]} + \text{[small square]} + \text{[small square]} + \text{[very small square]}$$

Cost Function

(Cost Function = Total Error)

- To get the overall error for the model, we compute the average of all squared errors – this gives us the Mean Squared Error (MSE), which is our cost function:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2$$

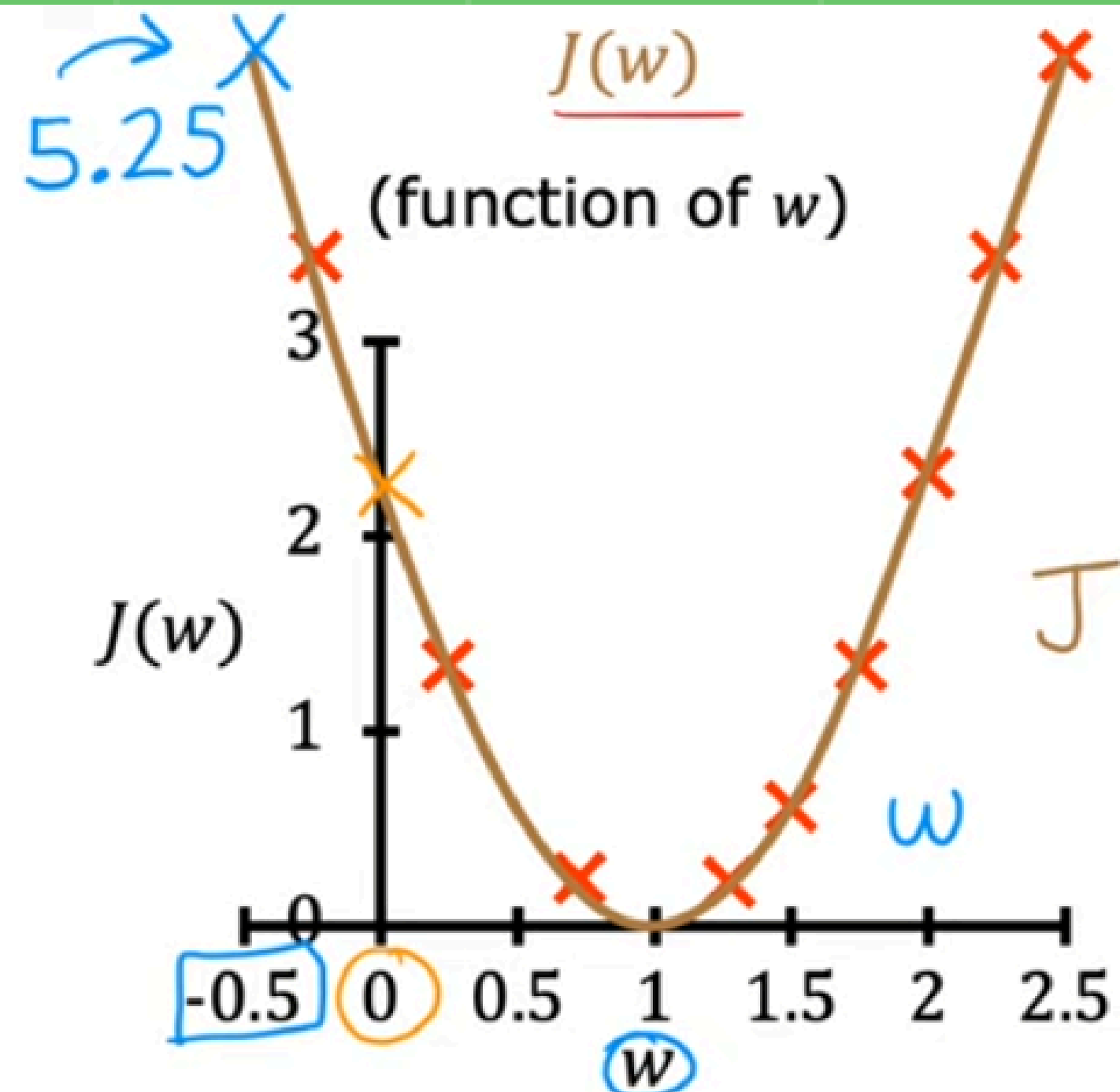
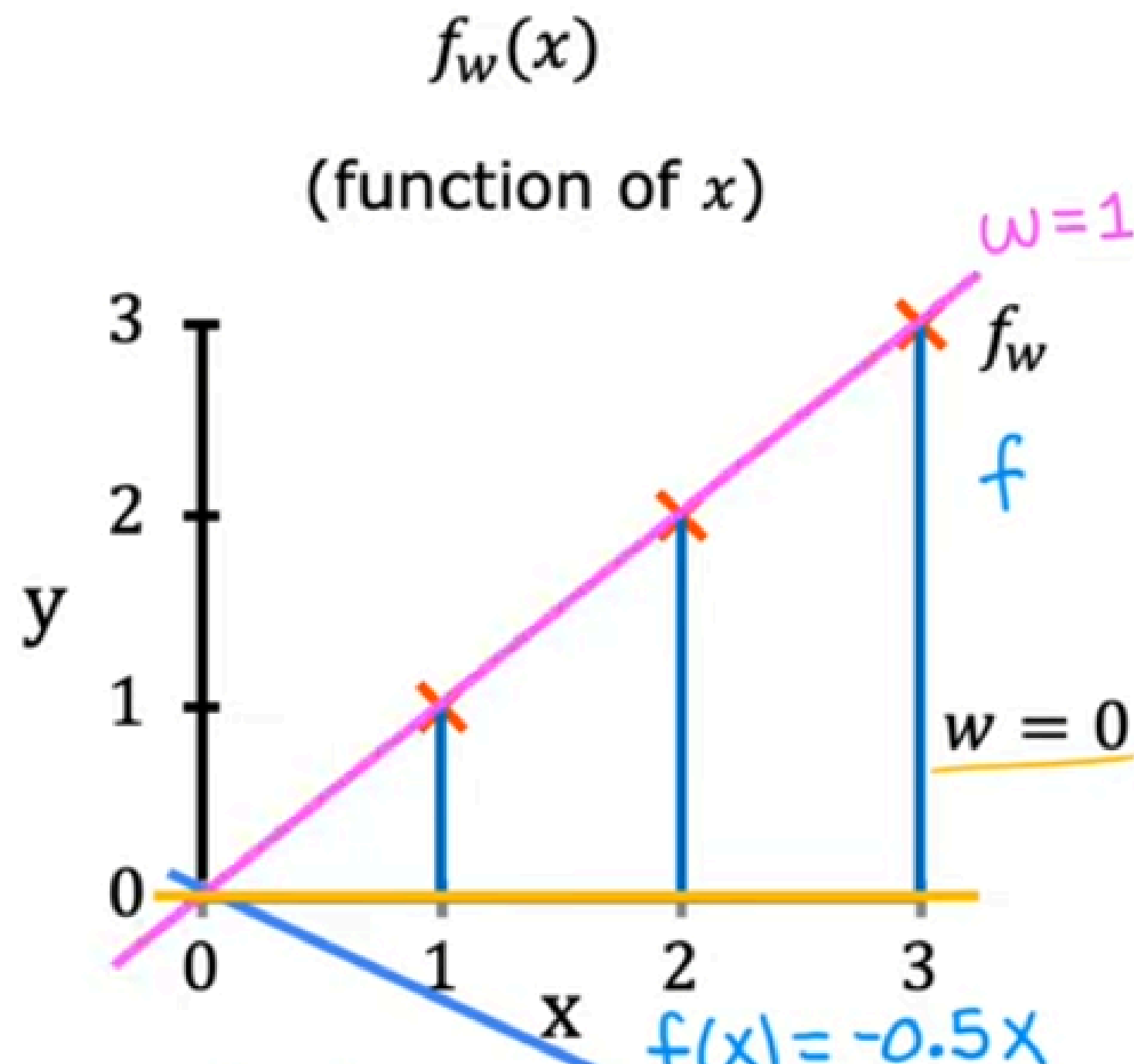
Cost Function

(Cost Function = Total Error)

- To get the overall error for the model, we compute the average of all squared errors – this gives us the Mean Squared Error (MSE), which is our cost function:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2$$

Cost Function

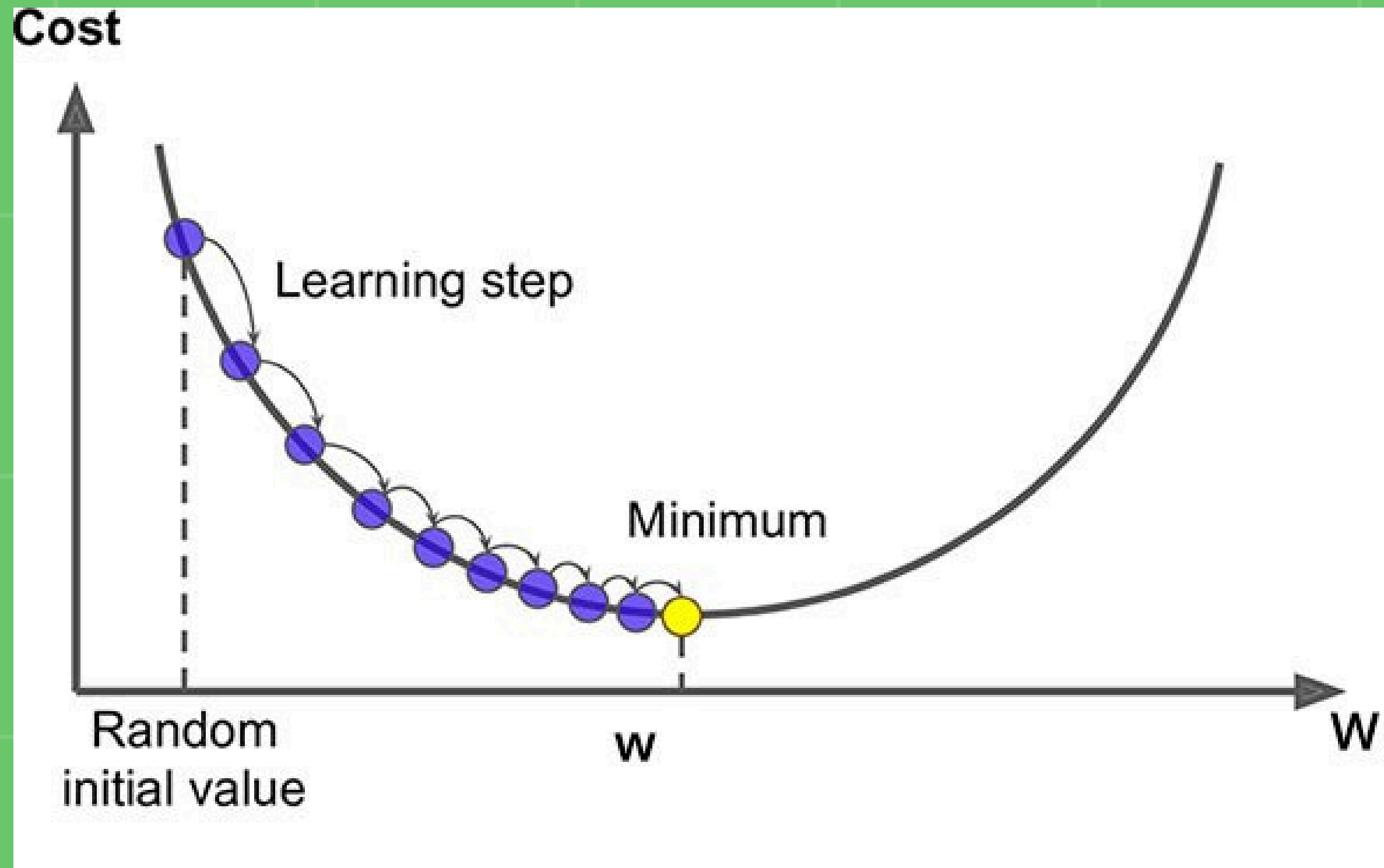


Gradient Descent

- An **optimization algorithm** used to minimize the cost function by adjusting the model's parameters in the direction that reduces the cost.

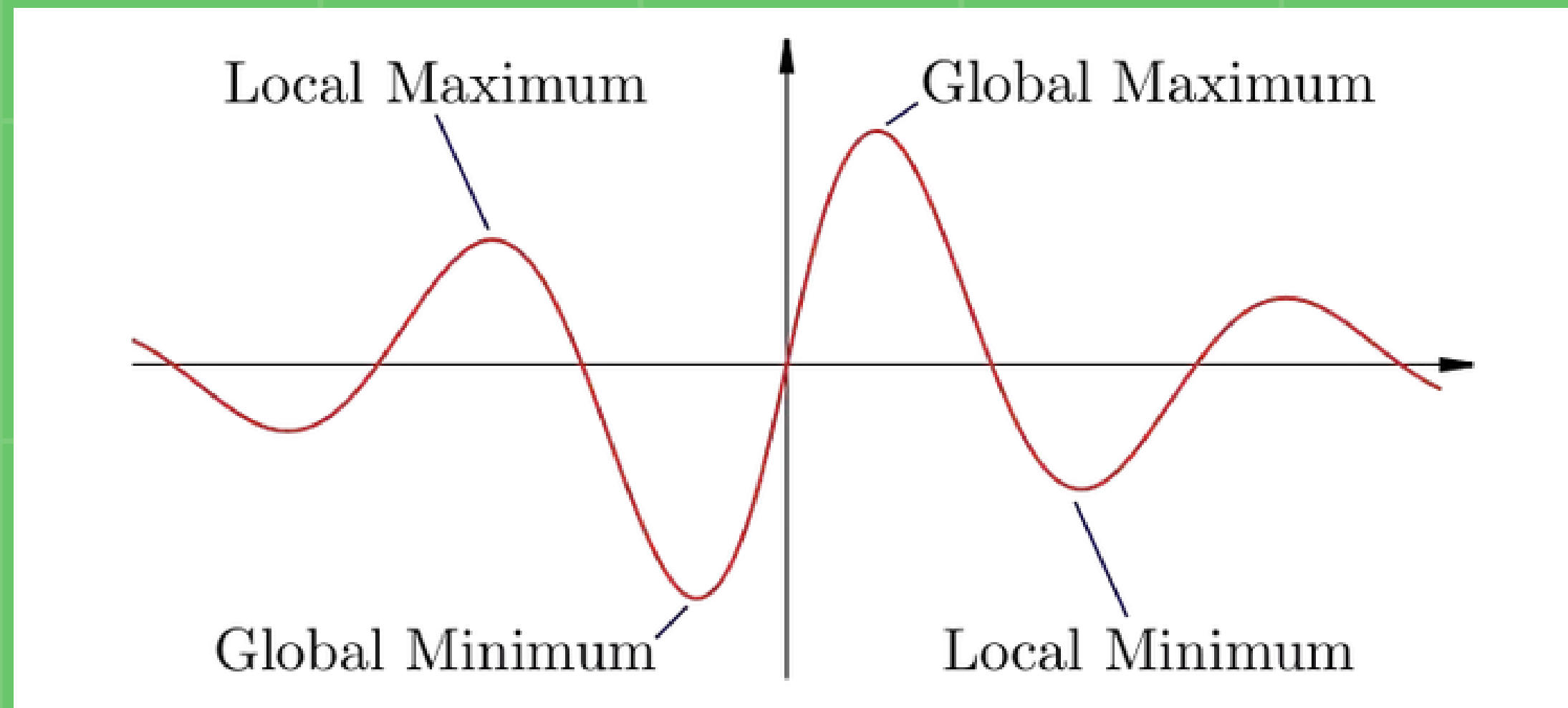
= Keep changing w , b to reduce cost function ($J(w, b)$) until we settle at or near a minimum.

Gradient Descent

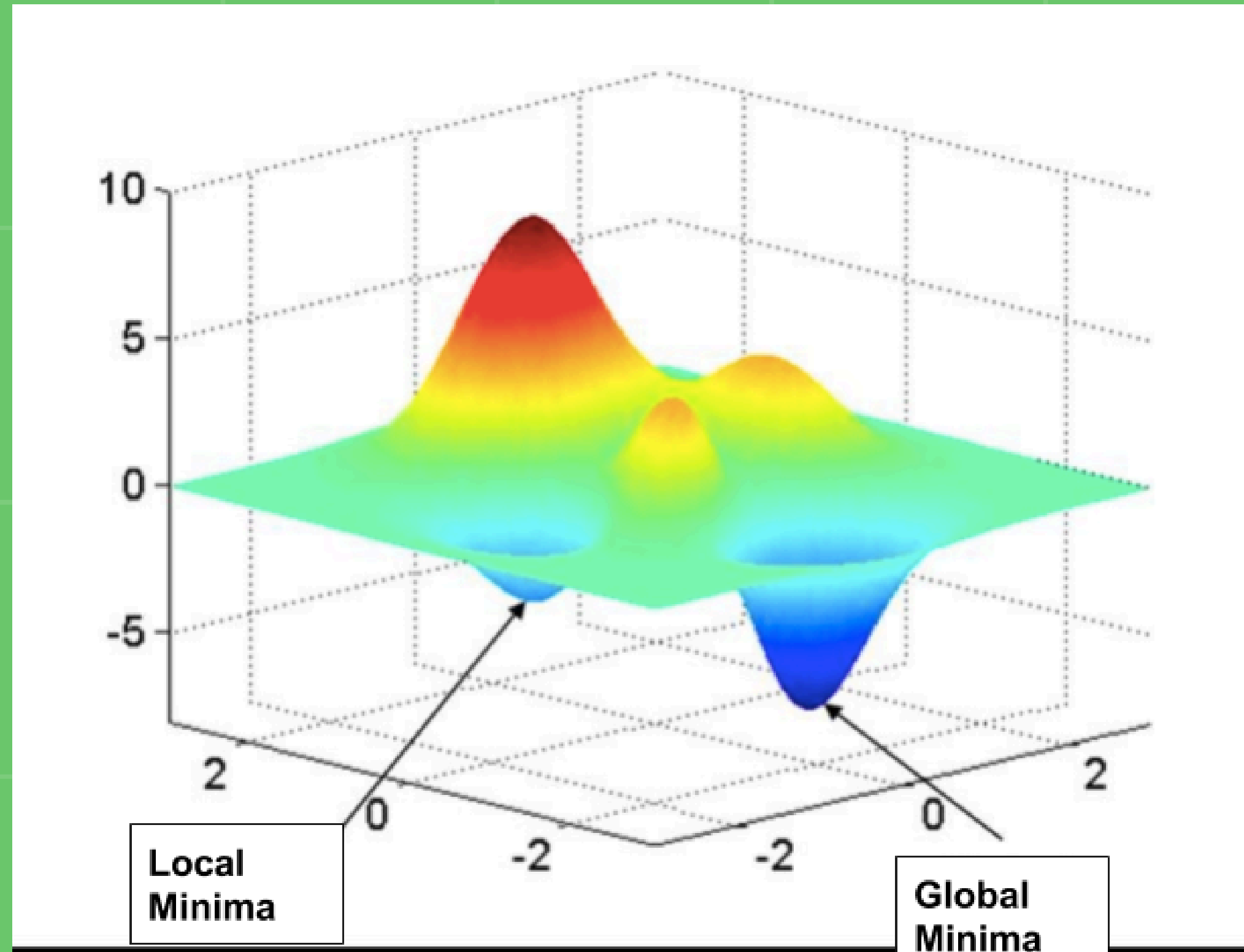


Global minima **VS** Local Minima

- **Global Minima:** the lowest point of the entire cost function (best possible set of parameters of the model)
- **Local Minima:** A point that is lower than its neighbours but not necessarily the lowest in the entire function.



Global minima **VS** Local Minima



Gradient Descent algorithm

Gradient descent algorithm

repeat until convergence {

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

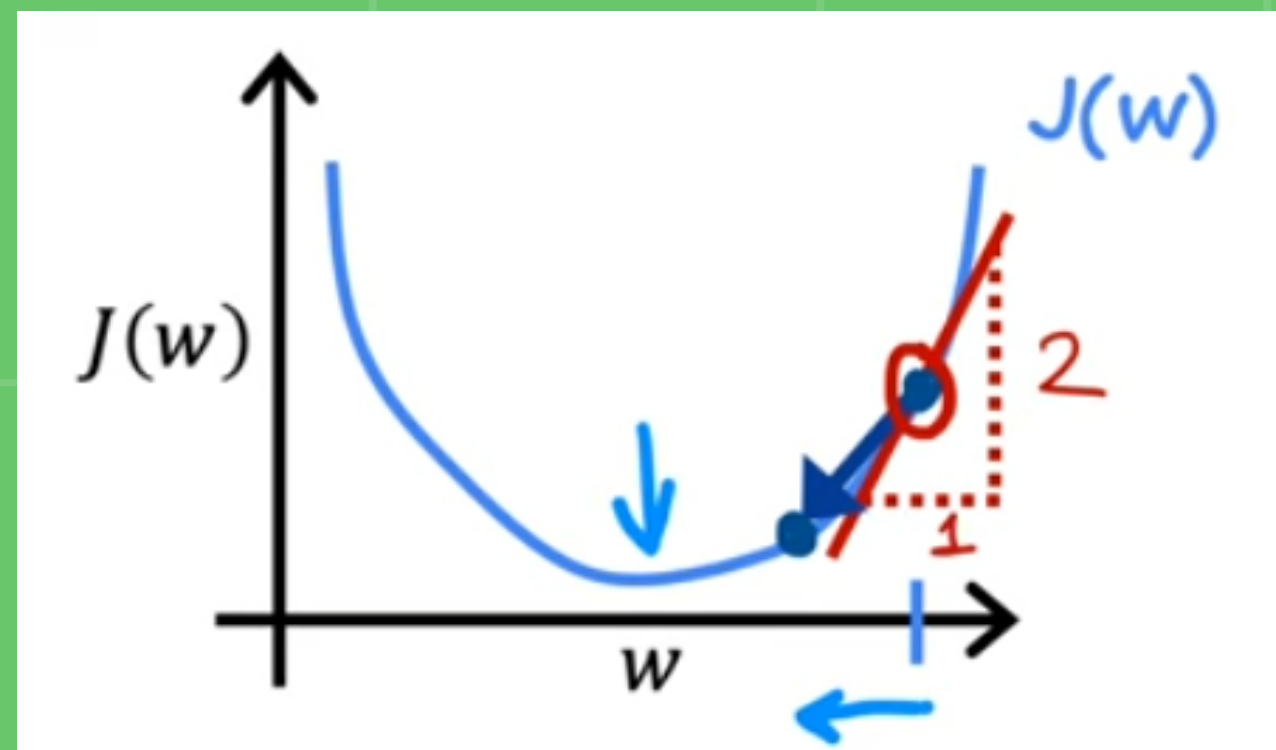
$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

}

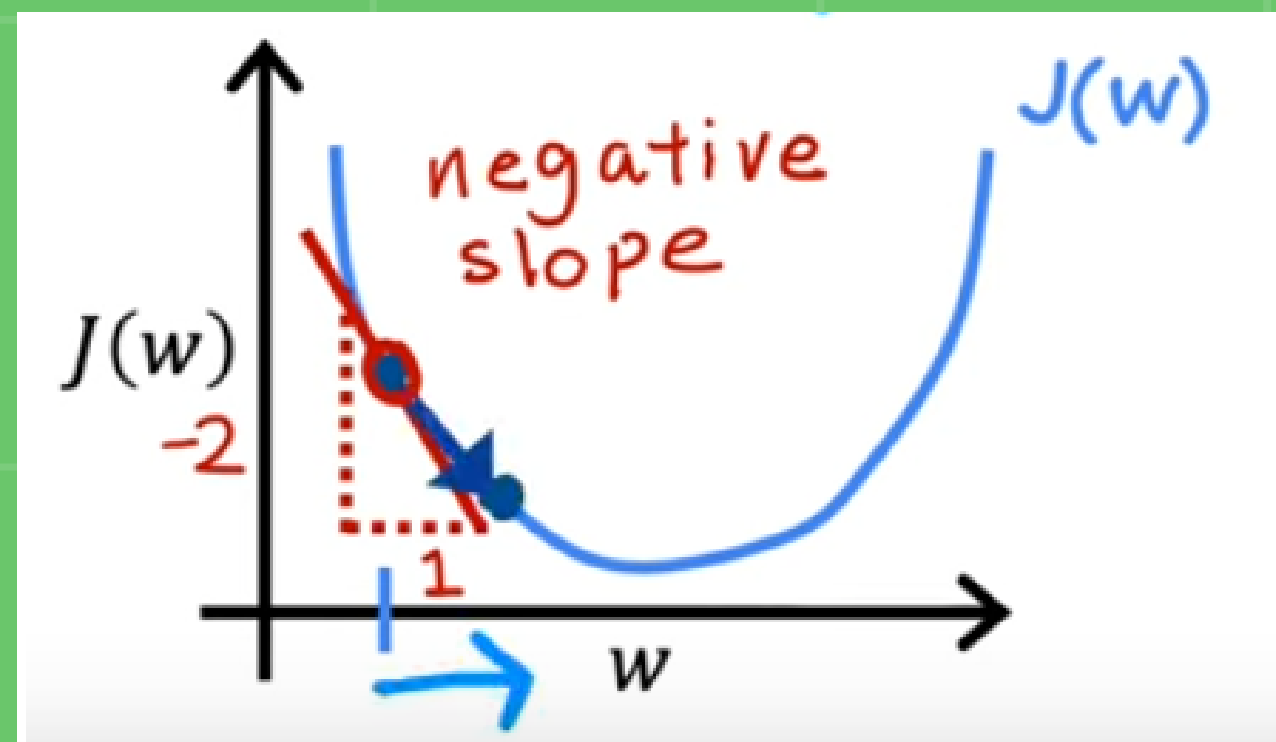
Gradient Descent algorithm

- What is convergence?
= We reach the convergence when w and b are not changing significantly anymore.
(when w and b are not changing anymore, that means we reached a global or a local minima, because we reached a point that is lower than the points near it).

Why we compute derivative?



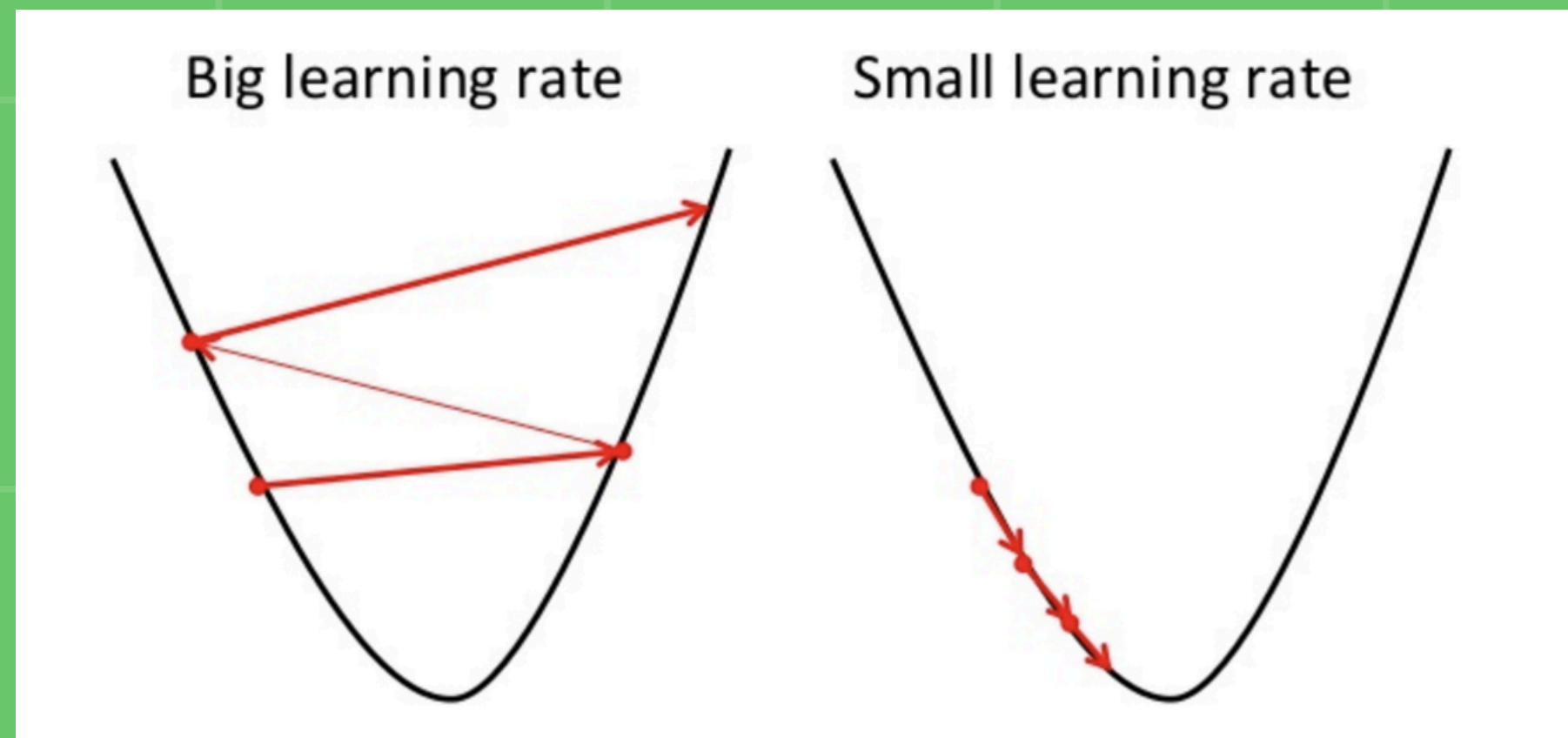
- When the slope is positive ($d/dw > 0$) then new value of w is smaller so we are decreasing to the left.



- When the slope is negative ($d/dw < 0$) then new value of w is bigger so we are increasing to the right.

Learning Rate

- It is one of the most important hyperparameters in the optimization algorithms.
- It controls how big a step the algorithm takes when updating the model parameters. (The speed at which algorithm learns)

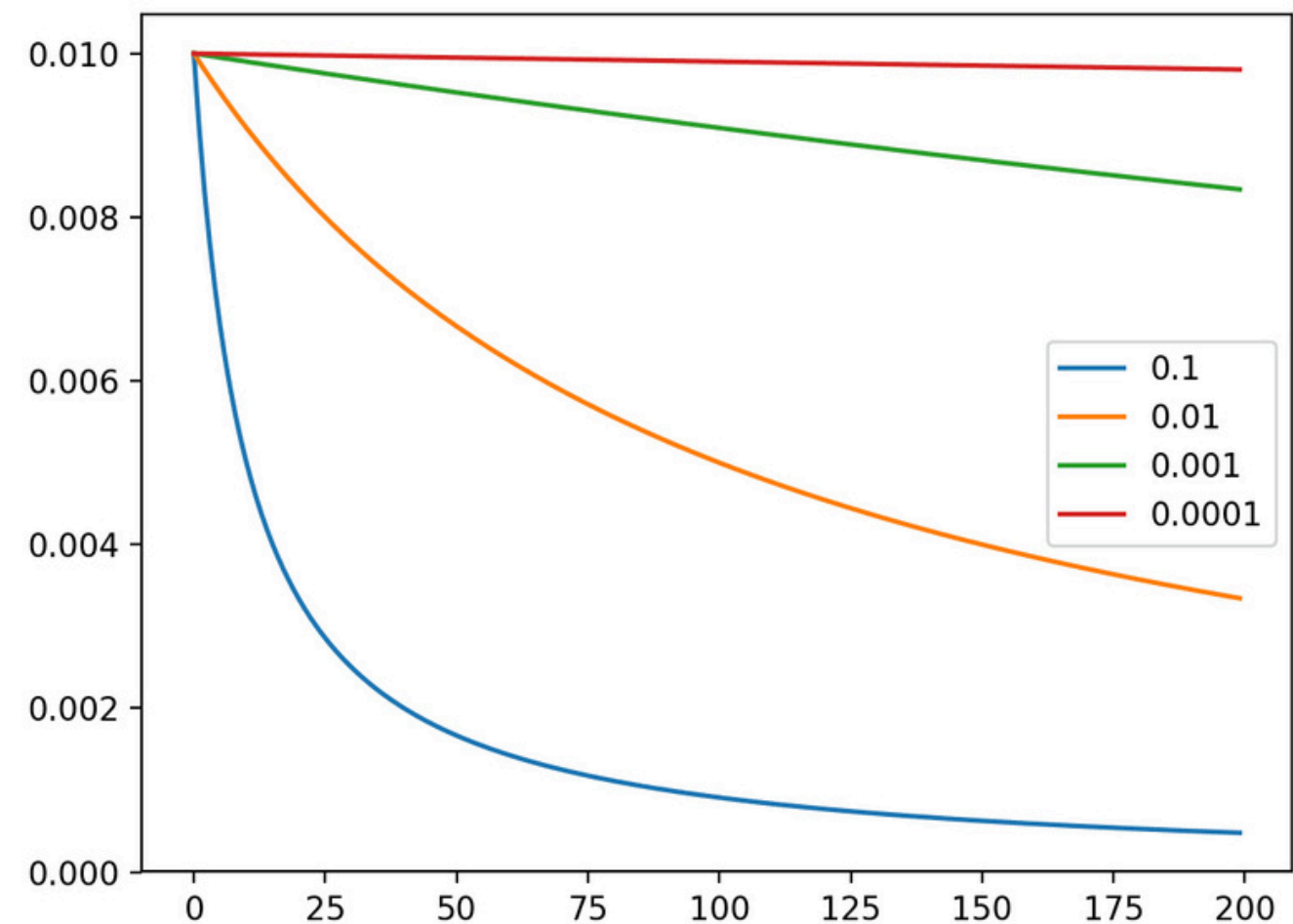


Learning Rate

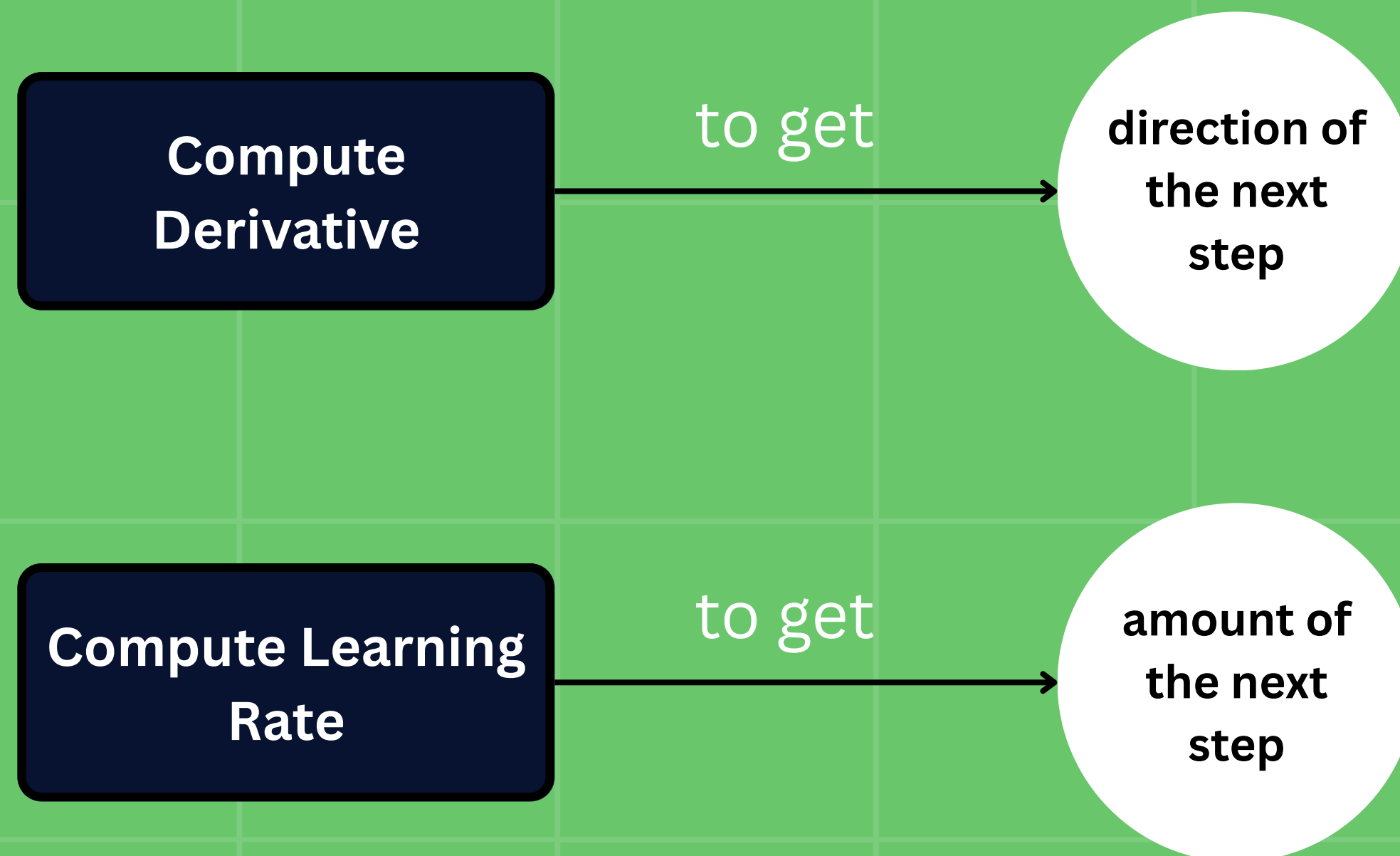
- If the learning rate is **too large**, the updates might be too big, causing the algorithm to overshoot the optimal solution. This can lead to divergence.
- If the learning rate is **too small**, the updates are very tiny, which makes the algorithm slow to converge or get stuck in a local minima.

How to Find the Optimal Learning Rate

- 1- Try a range of learning rate and track the loss function
- 2- Plot loss vs. learning rate to see where the loss decreases fastest before it spikes.



So, In Gradient Descent

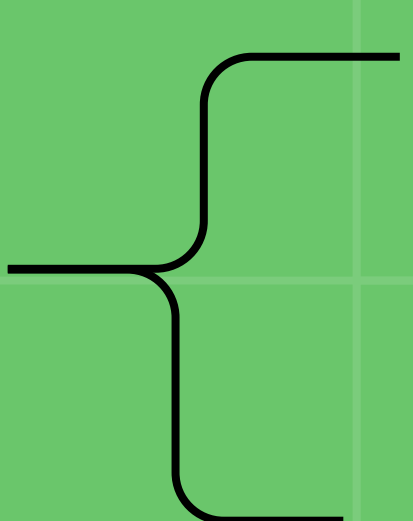


Multi-Valued Linear Regression

- Linear regression when there is more than one feature.

$$= w1*x1 + w2*x2 +...+ wn*xn + b$$

Parameters


$$\begin{bmatrix} w1 \\ w2 \\ \vdots \\ wn \end{bmatrix}$$

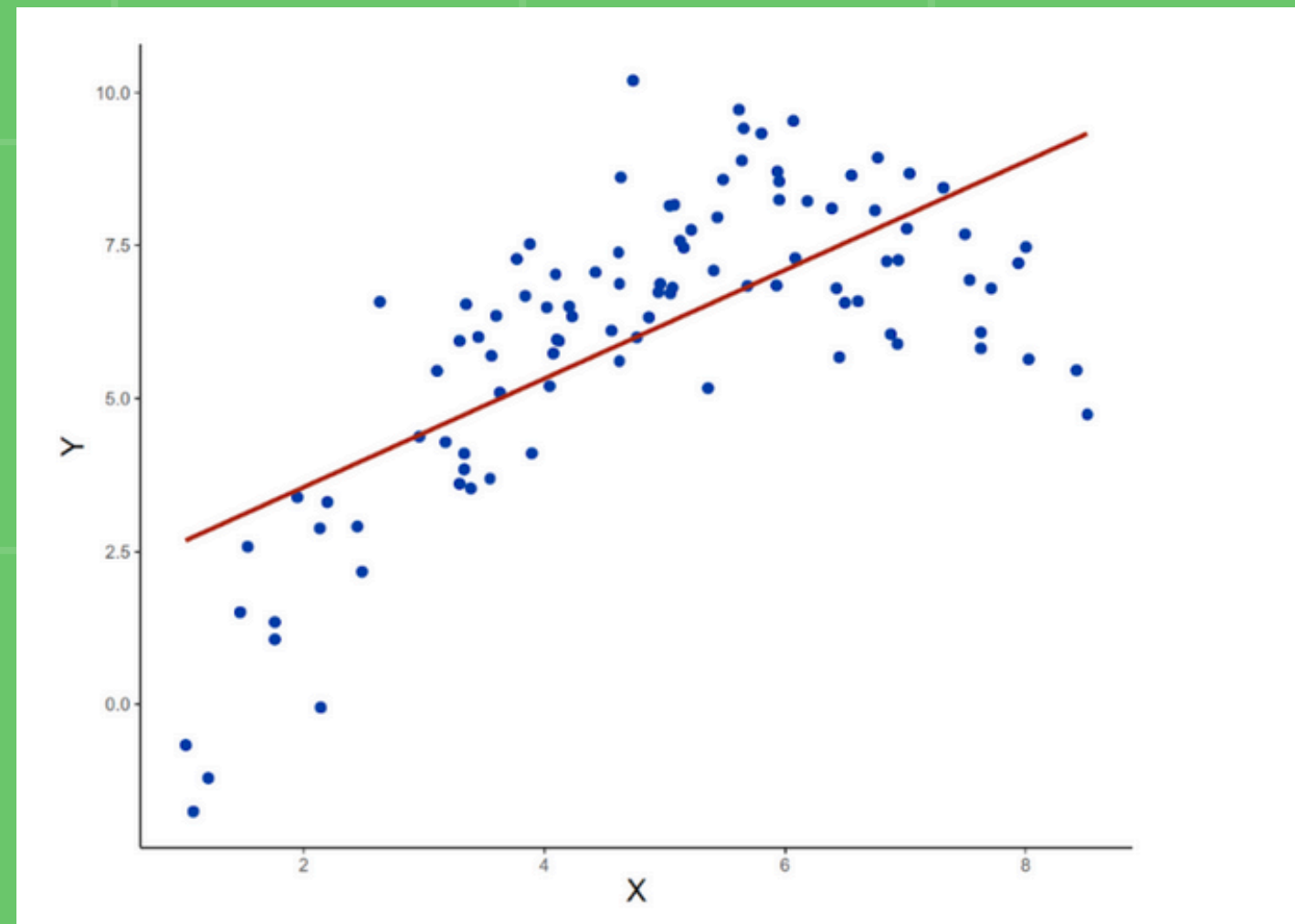
$$f(x) = \text{np.dot}(x, w) + b$$

Polynomial Regression

- Polynomial regression is a type of linear regression where the relationship between the input variable and the output is modeled as an n -th degree polynomial.
- It allows us to capture curved (non-linear) patterns in the data using a linear model on transformed features.

Polynomial Regression

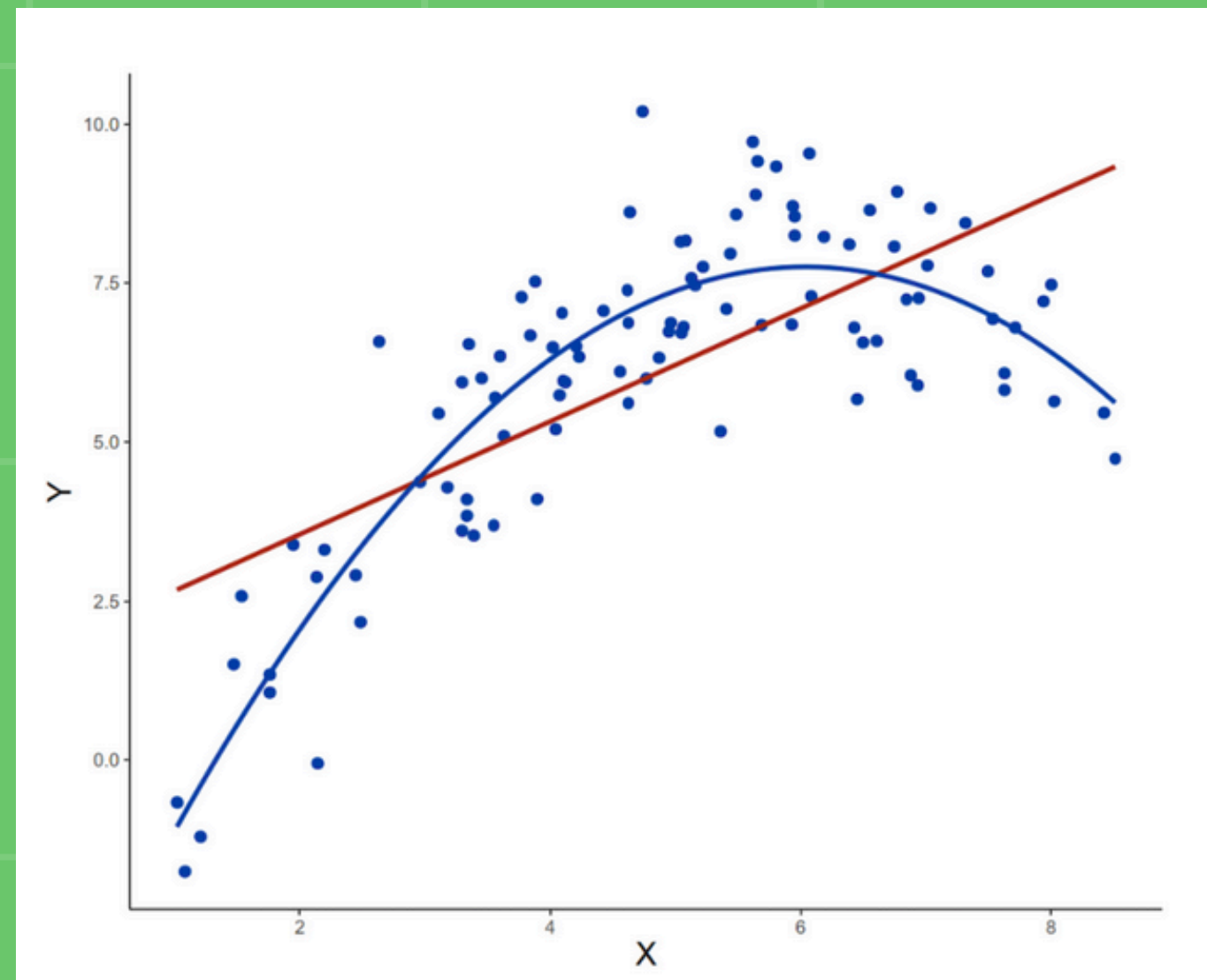
- Notice here that the linear line does not fit the data properly.



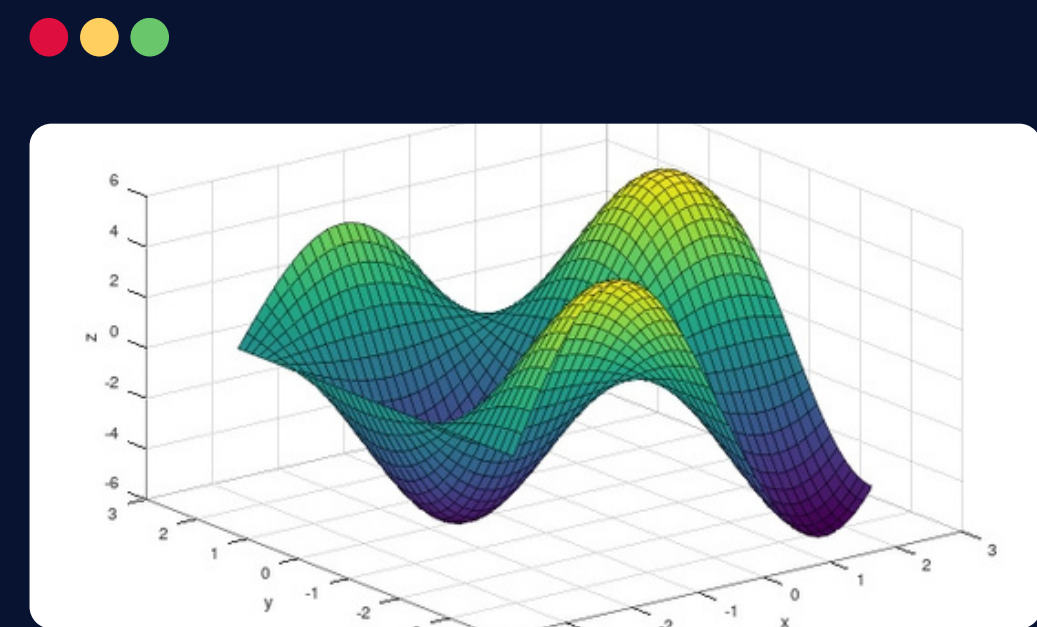
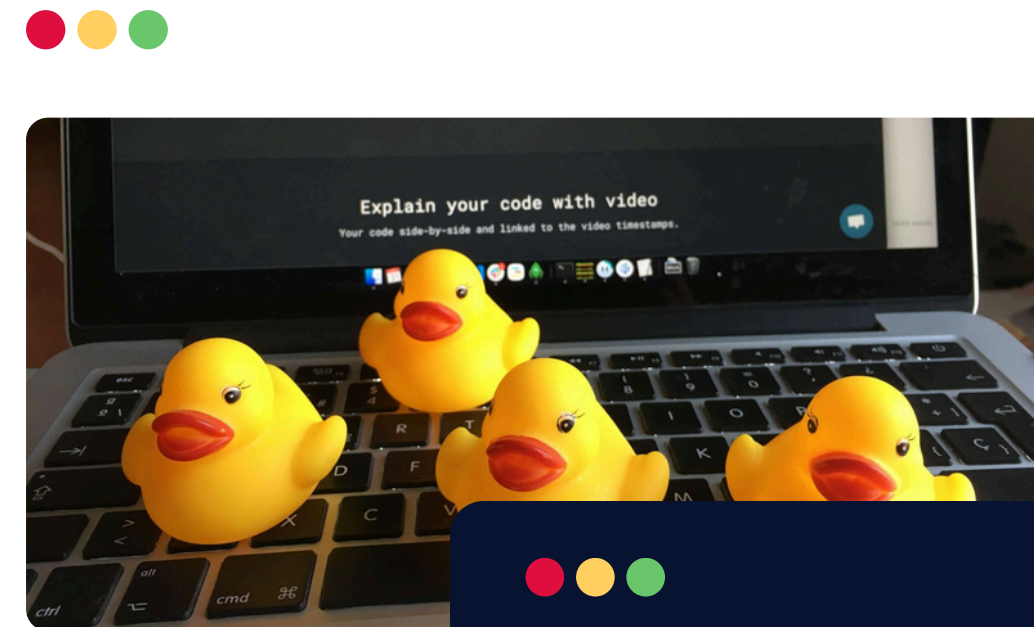
$$Y \sim a + bX + \epsilon$$

Polynomial Regression

- But when we increase the equation to a 2nd-degree polynomial, it models the curve better and gives us a more accurate fit.



$$Y \sim a + bX + bX^2 + \epsilon$$



Thank You

created by:
Ahd Ehab